

Document Technique : Explication du But et du Fonctionnement de la Transaction

1. But de la transaction

Le but de cette transaction est de garantir l'intégrité des données lors du processus de réservation de billets de cinéma. Elle permet de :

- Vérifier la disponibilité des sièges avant de procéder à leur blocage.
- Bloquer les sièges sélectionnés pour éviter qu'ils ne soient réservés par d'autres utilisateurs.
- Créer une nouvelle réservation dans la base de données.
- Mettre à jour les informations associées , telles que le nombre de sièges disponibles pour la séance.
- Assurer l'atomicité des opérations : Si une étape échoue (par exemple, si un siège n'est plus disponible), toutes les modifications précédentes sont annulées pour éviter des incohérences.

2. Fonctionnement de la transaction

La transaction est structurée en plusieurs étapes :

Étape 1 : Vérification de la disponibilité des sièges

- Une requête SQL vérifie que les sièges sélectionnés (**A1**, **A2**) sont disponibles (**IsAvailable = true**).
- Si un ou plusieurs sièges ne sont pas disponibles, une exception est levée, et la transaction est annulée.

Étape 2 : Blocage des sièges

- Les sièges sélectionnés sont marqués comme indisponibles (**IsAvailable = false**) pour éviter qu'ils ne soient réservés par d'autres utilisateurs.

Étape 3 : Mise à jour du nombre de sièges disponibles

- Le nombre de sièges disponibles pour la séance est décrémenté pour refléter les sièges bloqués.

Étape 4 : Création de la réservation

- Une nouvelle entrée est insérée dans la table **Reservations** avec les informations suivantes :
 - Identifiant de l'utilisateur (**AppUserId**).
 - Identifiant de la séance (**Showtimeld**).
 - Nombre de sièges réservés (**NumberOfSeats**).

- Prix total calculé (**TotalPrice**).
- QR Code généré pour la réservation (**QrCode**).
- Statut initial de la réservation (**Confirmed**).

Étape 5 : Association des sièges à la réservation

- Les sièges sélectionnés sont associés à la réservation via la table de jointure **SeatReservations**.

Gestion des erreurs

- Si une erreur survient à n'importe quelle étape (par exemple, si un siège n'est plus disponible), la transaction est annulée (**ROLLBACK**), et toutes les modifications précédentes sont réverties.
- Cela garantit que les données restent cohérentes, même en cas de problème technique ou d'erreur.

3. Pourquoi une transaction est nécessaire ?

Une transaction est essentielle pour garantir que toutes les opérations liées à la réservation sont exécutées ensemble. Sans transaction, il serait possible d'avoir des incohérences dans les données, par exemple :

- Des sièges bloqués sans réservation associée.
- Une réservation créée sans mise à jour des sièges ou du nombre de sièges disponibles.
- Une surréservation de sièges due à des conflits concurrents.

La transaction garantit que :

- Soit toutes les opérations réussissent, et les données sont mises à jour de manière cohérente.
- Soit aucune opération n'est appliquée, et les données restent dans leur état initial.

4. Exemple concret de flux de travail

Supposons qu'un utilisateur souhaite réserver deux sièges (**A1**, **A2**) pour une séance spécifique. Voici comment la transaction fonctionne :

1. Vérification des sièges : La base de données vérifie que les sièges **A1** et **A2** sont disponibles.
2. Blocage des sièges : Les sièges sont marqués comme indisponibles.
3. Mise à jour de la séance : Le nombre de sièges disponibles pour la séance est décrémenté de 2.

4. Création de la réservation : Une nouvelle réservation est créée dans la table **Reservations**.
5. Association des sièges : Les sièges **A1** et **A2** sont associés à la réservation via la table de jointure **SeatReservations**.

Si une erreur survient à n'importe quelle étape (par exemple, si un autre utilisateur réserve **A1** en même temps), toutes les modifications sont annulées, et les sièges sont libérés.

5. Indices et performances

Pour garantir des performances optimales :

- Index sur **Seat.SeatNumber** : Facilite les recherches de sièges par numéro.
- Index sur **Showtime.ShowtimeId** : Accélère les recherches de séances spécifiques.
- Index composite sur **SeatReservations** : Améliore les performances des jointures entre **Reservations** et **Seats**.

6. Contraintes de validation

Des contraintes au niveau de la base de données garantissent l'intégrité des données :

- Contrainte **UNIQUE** sur **Seat.SeatNumber** : Assure qu'un numéro de siège est unique pour une salle donnée.
- Contrainte **CHECK** sur **Seat.IsAvailable** : Garantit que la valeur de **IsAvailable** est toujours valide.
- Trigger pour libérer automatiquement les sièges après un délai : Permet de libérer les sièges bloqués mais non confirmés après un certain délai.